

AN IMPROVED ALGORITHM FOR CHECKING THE COLLATZ CONJECTURE FOR ALL $n < 2^N$

VIGLEIK ANGELTVEIT

ABSTRACT. We describe a new algorithm for verifying the Collatz conjecture for all $n < 2^N$ for some fixed N . The algorithm takes less than twice as long to verify convergence for all $n < 2^{N+1}$ as it does to verify convergence for all $n < 2^N$.

1. INTRODUCTION

Let $n \geq 1$ be a natural number, and define the Collatz function $C(n)$ and the modified Collatz function $T(n)$ by

$$C(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n + 1 & \text{if } n \text{ is odd} \end{cases} \quad T(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ (3n + 1)/2 & \text{if } n \text{ is odd} \end{cases}$$

The Collatz conjecture states that for any n , by applying C enough times we eventually reach the orbit $1 \mapsto 4 \mapsto 2 \mapsto 1$. Equivalently, by applying T enough times we eventually reach the orbit $1 \mapsto 2 \mapsto 1$.

A proof of the Collatz conjecture appears to be out of reach with current techniques, but considerable computer resources have been devoted to verifying the Collatz conjecture for all $n < K$ for increasingly large K . The most recent progress on the subject is described in [3], where Barina verifies convergence for all $n < 2^{71}$, and we will refer to this paper repeatedly for the current state of the art.

The goal of the current paper is to describe a more efficient algorithm for verifying the Collatz conjecture up to some K . Unlike previous algorithms, the algorithm presented in this paper becomes more efficient for higher K in the sense that the fraction of integers in $[1, 2^{N+1}]$ that must be checked is (usually) smaller than the fraction of integers in $[1, 2^N]$ that must be checked. Indeed, as $N \rightarrow \infty$ the fraction of integers in $[1, 2^N]$ that must be checked approaches 0. The number of integers that must be checked does of course still go to infinity. For the rest of the paper we fix N , and explain how to check that any $n < 2^N$ converges.

The key idea is to consider all n with the same k least significant bits (when written in binary) at the same time, and then do a recursive search where we add one bit at a time.

As a further optimisation, we precompute some bitvectors of length 2^B and use one of them to add the A most significant bits in one go by looking B steps ahead, for some chosen constants A and B with $B > A$. This makes the tail end of the recursive search faster at the same time as it excludes more cases.

At a high level this can be described as in Algorithm 1.

Algorithm 1: a high level overview of the algorithm to verify convergence for all $n < 2^N$.

- (1) By recursively adding more bits, starting with the least significant bit, exclude all $n_0 < 2^{N-A}$ for which $T^k(n_0) < n_0$ for some $0 < k \leq N - A$, and at least some $n_0 < 2^{N-A}$ for which there exists an $n_1 < n_0$ with $T^k(n_0) = T^\ell(n_1)$. Simultaneously compute $T^{N-A}(n_0)$ for the remaining possible n_0 .
 - (2) Use precomputed bitvectors to exclude all $n = n_0 + a2^{N-A} < 2^N$ with $a = 0, \dots, 2^A - 1$ for which $T^k(n) < n$ for some $N - A < k \leq N - A + B$ while also excluding all $n \equiv 2, 4, 5, 8 \pmod 9$ and at least some $n < 2^N$ for which there exists an $n_1 < n$ with $T^k(n) = T^\ell(n_1)$. Simultaneously compute $T^{N-A}(n)$ for the remaining values of n .
 - (3) For the remaining values of $n < 2^N$, compute $T^k(n)$ for $k > T^{N-A}(n)$ until this value falls below n .
-

The exclusion of $n \equiv 2, 4, 5, 8 \pmod 9$ in Step (2) comes from the fact that in those cases $n = T^\ell(n_1)$ for some $n_1 < n$. To also detect n for which $T^k(n) = T^\ell(n_1)$ for some $n_1 < n$ we add two more sieves to Steps (1) and (2). See Section 2 below for a description of all the sieves we use.

To verify convergence in a larger range than Barina we can, to start with, choose $N = 72$. Or we could be more ambitious and choose a larger N . If we can verify convergence for all $n < 2^{75}$ we can then use the techniques in [4] to show that a nontrivial cycle, if one exists, must have length at least 2302268119908. That is a 6.5-fold improvement on the result in their paper. Verifying convergence for $n < 2^{74}$ might be enough if combined with another search.

In our computer implementation we chose $A = 6$ for the CPU version and $A = 10$ for the GPU version, and $B = 24$ for both versions. The choice is based on benchmarking different values.

In [3, Section 3.2] Barina states that using a 3^k sieve (with $k = 2$) is a time-critical operation. We implement the 3^2 sieve, which we will refer to as the mod 9 Preimage Sieve, by taking a bitwise **and** of one of 9 precomputed bitvectors of length 2^A and a part of one of the bitvectors of length 2^B mentioned above. This reduces the number of mod 9 calculations by a large factor and makes the mod 9 sieve very cheap to implement.

As we will explain below, our algorithm is equivalent to using a precomputed sieve of size 2^{N-A+B} , where the precomputed bitvectors have size 2^B , as well as the Mod 9 Preimage Sieve and two other sieves. By this we mean that the fraction of integers we actually need to consider is the same as the fraction one would get by using such a large precomputed sieve plus the other sieves.

If we set $N = 72$, $A = 10$, and $B = 24$, this means that our algorithm is equivalent to using a sieve of size 2^{86} plus the three other sieves. Moreover, Step (2) computes $T^{N-A}(n)$, which saves time when checking those numbers that survive the initial sieves. This compares to using a sieve of size 2^{34} from [3, Table 4]. That is close to the limit of what is practical to store in memory; one advantage of our algorithm is that we avoid storing the large sieve.

One disadvantage of our algorithm is that it does not proceed linearly. It can, for example, be broken up into cases based on the last k bits for a convenient value of k , but it cannot easily be adapted to check an interval $[K, L]$ of integers. This makes

a direct comparison with Barina’s work [3] difficult, as he breaks the computation up into intervals of length 2^{40} .

1.1. A computer implementation. We have implemented our algorithm both for running on a CPU (in Rust) using native 128-bit integer types, and for running on a GPU (in CUDA) using multiple 32-bit integers to represent a 128-bit integer value. The Rust program is about 300 lines of code while the CUDA program is about 550 lines.

The GPU version is, unsurprisingly, much faster than the CPU version. The running time appears to be significantly faster than other computer programs described in the literature. We estimate that it would take approximately 400 years of CPU time to verify the Collatz conjecture for all $n < 2^{72}$. Running on a GPU it would take somewhere in the range of 2.5-20 years, depending on the GPU. Here the 20 year estimate is based on benchmarking with a Nvidia RTX 3060 GPU. The calculation can of course be split up into cases to be processed in parallel. We have made both versions available at <https://github.com/vigleik0/collatz>.

We used the CPU version of our program to check all integers up to 2^{40} for convergence, using the appropriate sieves. It explicitly checked 888 966 177 integers, or about 0.081% of all natural numbers up to 2^{40} . The program took 25 seconds, which translates to about $2^{35.4}$ integers per second, on a single AMD Zen 3 core running at 4.6GHz. We then ran the same program to check all integers up to 2^{50} . This time it explicitly checked 509 760 079 810 integers, which is about 573 times as many. This time the calculation took 12 161 seconds, or about $2^{36.4}$ integers per second. In other words, it took about 2^9 times as long to check 2^{10} times as many integers.

This is significantly faster than the 217s per 2^{40} numbers in [3, Table 7], especially when extrapolated to checking up to 2^N for some $N > 71$.

We used the GPU version to check all integers up to 2^{50} . The program took 451s on a Nvidia RTX 3060 with 3584 CUDA cores. This translates to about $2^{41.2}$ integers per second. We then checked all integers up to 2^{60} . This time the program took 3 days, which translates to about $2^{42.0}$ integers per second. This illustrates how the algorithm becomes better when checking a larger range. This GPU is obviously not state of the art: A Nvidia RTX 5090 has about 8 times the 32-bit integer arithmetic performance, and we assume it will speed up the program by a similar factor.

We estimate that with our algorithm we would have to check less than 0.02% of all natural number up to 2^{72} .

To check for correctness, we had our program output all starting values n in the appropriate range for which $T^k(n)$ reached a large value. We compared this to the table of path records in [1], and verified that our program detects all the path records with a starting value below 2^{60} .

To make a potential distributed computation possible, we have split the problem into either 1 390 or 328 718 separate cases of (at least conjecturally) similar running time.

2. SOME SIEVES

In this section we give some background material and discuss four different sieves. It is convenient to use the function T instead of the function C , because we get to divide by 2 exactly once each time.

Lemma 2.1 ([6]). *The sequence of even and odd applications of T in the k -fold composite $T^k(n)$ only depends on the last k bits of the binary representation of n .*

For example, because $3 = (11)_2$ and $11 = (1011)_2$ share the same last 3 bits we can compute $T^3(3)$ and $T^3(11)$ using the same sequence of even and odd applications of T : odd, odd, even. For 3 we get $3 \mapsto 5 \mapsto 8 \mapsto 4$ and for 11 we get $11 \mapsto 17 \mapsto 26 \mapsto 13$.

Definition 2.2. We let $f_k(n)$ be the number of odd applications of T to get to $T^k(n)$. Equivalently, $f_k(n)$ is the number of odd integers in $\{n, T(n), \dots, T^{k-1}(n)\}$.

For example, $f_3(11) = 2$ since 2 of the 3 integers $\{11, T(11), T^2(11)\} = \{11, 17, 26\}$ are odd. By Lemma 2.1, $f_k(n)$ depends only on the last k bits of n .

Lemma 2.3 ([6]). *Suppose $n = n_0 + a2^k$ for some $a \geq 0$ and that $T^k(n_0)$ has $f = f_k(n_0)$ odd applications. Then*

$$T^k(n) = T^k(n_0) + 3^f a.$$

For example, we can take $n_0 = 3$ and $n_1 = 3 + 1 \cdot 2^3 = 11$. Since $T^3(n_0)$ has $f = 2$ odd applications, $T^3(11) = T^3(3) + 3^2$. This has been used by various authors to implement a lookup-table for computing $T^k(n)$ in a single step by looking at the last k bits of n , and we used this in our CPU implementation to do $k = 16$ steps at a time in Step (3) of Algorithm 1. For our GPU implementation we found that it was better to avoid constantly looking up values in a table, and we instead used [2, Algorithm 1].

We immediately get the following:

Corollary 2.4. *Suppose $T^k(n_0)$ has $f = f_k(n_0)$ odd applications. If $T^k(n_0)$ is even then*

$$T^{k+1}(n_0 + 2^k) = \frac{3(T^k(n_0) + 3^f) + 1}{2},$$

and if $T^k(n_0)$ is odd then

$$T^{k+1}(n_0 + 2^k) = \frac{T^k(n_0) + 3^f}{2}.$$

This lets us compute both $T^{k+1}(n_0)$ and $T^{k+1}(n_0 + 2^k)$ from $T^k(n_0)$, so we can compute $T^M(n)$ for all $n < 2^M$ inductively by adding one bit at a time starting from the least significant bit. We keep track of the number f of odd applications of T , and use a precomputed table of powers of 3.

It is perhaps interesting to note that for any natural number $t < 2^M$, if we interpret the k 'th bit in the binary representation of t being 0 as the k 'th application of T being even and the k 'th bit being 1 as the k 'th application of T being odd then there is exactly one natural number $n < 2^M$ for which repeated application of T gives that pattern of even and odd. This defines a somewhat mysterious bijection $\mathbb{Z}/2^M \rightarrow \mathbb{Z}/2^M$. By letting M go to infinity this gives a bijection from the 2-adic integers to itself.

We now describe the different sieves we use.

2.1. The Descent Sieve. This sieve relies on the following result:

Theorem 2.5. *Suppose $n_1 = n_0 + a2^k$ for some $a \geq 0$ and that $T^k(n_0) < n_0$. Then $T^k(n_1) < n_1$ as well.*

Proof. By Lemma 2.1, we use the same sequence of even and odd applications of T to compute $T^k(n_0)$ and $T^k(n)$. If m is even then $\frac{T(m)}{m} = \frac{1}{2}$ while if m is odd then $\frac{T(m)}{m} = \frac{3}{2}(1 + \frac{1}{3m})$. Since $n_1 \geq n_0$ and both have the same sequence of even and odd applications of T , we have $\frac{T^k(n_1)}{n_1} \leq \frac{T^k(n_0)}{n_0}$, and the result follows. $\square$

Given some $n_0 < 2^k$, the Descent Sieve works by checking if $T^k(n_0) < n_0$. If so, we exclude all n given by adding more significant bits to n_0 . This works because by induction we can assume that we have already verified convergence for $T^k(n_1)$ for any n_1 with the same last k bits as n_0 .

The basic idea behind this sieve is of course standard, and various authors have used it to precompute a table of mod 2^M values that need to be considered for some M . But the idea of inductively adding more significant bits appears to be new.

2.2. The mod 9 Preimage Sieve. This sieve relies on the following result:

Lemma 2.6. *Suppose $n \equiv 2 \pmod{3}$. Then $n = T((2n - 1)/3)$. Similarly, if $n \equiv 4 \pmod{9}$ then $n = T^3((8n - 5)/9)$.*

This means that if $n \equiv 2, 4, 5$ or $8 \pmod{9}$ then n is on the path of either $(2n - 1)/3$ or $(8n - 5)/9$, so by induction it does not need to be considered.

This is another standard sieve. See for example [3, Section 3.3].

2.3. The Path-Merging Sieve. We can also use Lemma 2.6 to prune a later part of the search tree.

Lemma 2.7. *Suppose $m = T^k(n)$ and $m \equiv 2 \pmod{3}$. Then the path of n joins the path of $(2m - 1)/3$, so if $(2m - 1)/3$ converges then so does n .*

This sieve works by checking if $m = T^k(n) \equiv 2 \pmod{3}$ and $(2m - 1)/3 < n$. If these conditions hold then by induction n does not have to be checked.

Note that if $n = n_0 + a2^k$ and $k \geq 1$ it follows from Lemma 2.3 that $T^k(n) \equiv T^k(n_0) \pmod{3}$, so this sieve only depends on the last k bits of n .

Lemma 2.8. *Suppose $m = T^k(n)$ has at least one odd step. Then $m \equiv 2 \pmod{3}$ if and only if there is an even number of even steps after the last odd step.*

For example, the path of $n = 15$ starts as $15 \mapsto 23 \mapsto 35 \mapsto 53 \mapsto 80 \mapsto 40 \mapsto 20$. Since there are 2 even steps from the last odd step $53 \mapsto 80$ to 20, we can conclude that $20 \equiv 2 \pmod{3}$ without doing the modular arithmetic calculation. Since $T(13) = 20$ we can then exclude 15 from consideration. Because this example had a total of 6 steps, it only depends on the last 6 bits of $n = 15$. So this sieve excludes any $n \equiv 15 \pmod{64}$. For example, it also excludes $n = 79$, even though $T^6(79) = 101 > 79$ and the path of 79 continues up for at least one more step.

There is a corresponding sieve that works whenever $m \equiv 4 \pmod{9}$, but it is less effective and we did not implement it.

Yet another version of this sieve works whenever $m \equiv 8 \pmod{9}$, because then $T^2((4m - 5)/9) = m$ so it applies when $m \equiv 8 \pmod{9}$ and $(4m - 5)/9 < n$. This sieve is also not very effective, so we did not implement it. More generally there is a version whenever $m \equiv -1 \pmod{3^k}$ for any k .

2.4. The Odd-Even-Even Sieve. This sieve relies on the following result:

Lemma 2.9 ([4, Lemma 9]). *Suppose there are $k > 0$ odd steps in the Collatz sequence following m and then at least 2 even steps. Then the Collatz sequence of m joins that of $(m - 1)/2$.*

The sieve works by checking if $m = T^k(n)$ is such that $(m - 1)/2 < n$. When that happens, we look for a sequence of odd steps followed by 2 even steps, and if such a sequence occurs we exclude n .

See Figure 2.1 for a summary of all four sieves.

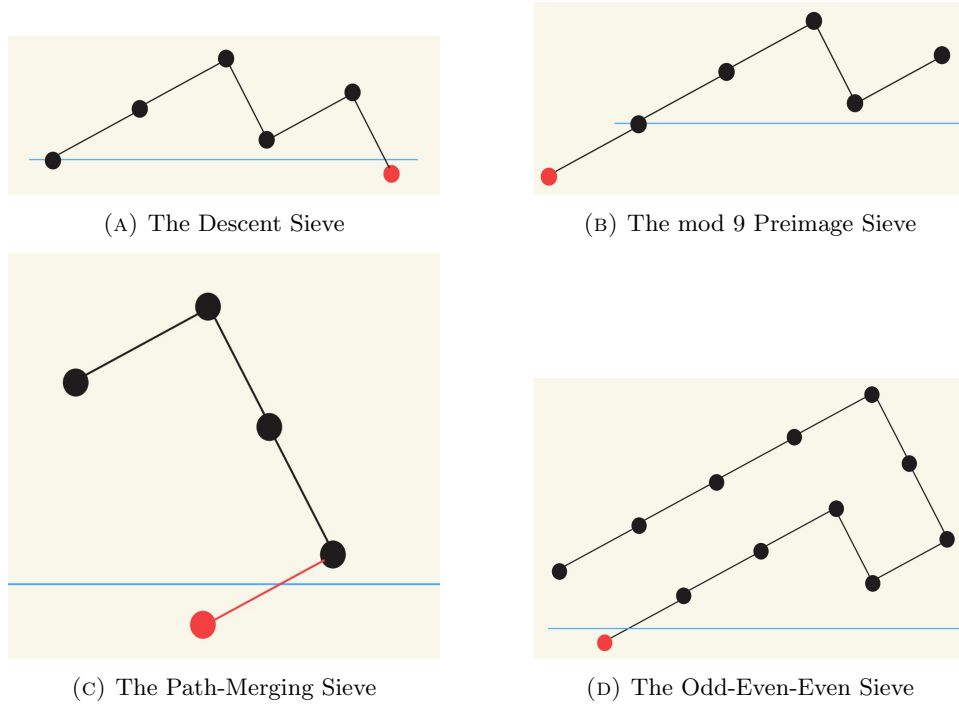


FIGURE 2.1. The blue line indicates the starting value n , and the red dot indicates the value $n_1 < n$ whose path joins that of n .

3. STEP (1) OF ALGORITHM 1

We now describe Step (1) of Algorithm 1 in more detail. This uses the Descent Sieve, the Path Merging Sieve, and the Odd-Even-Even sieve.

A limited version of the Descent Sieve has been used many times before. For example, in [5] the authors used this idea to precompute a sieve of size 2^k for k as large as 37, and in [3] the author used this idea to precompute a sieve of size 2^{34} (for use on a CPU) or size 2^{24} (for use on a GPU).

We use this algorithm for the least significant $M = N - A$ bits of n , and the method from the next section for the last A bits, for some A between 6 and 10. We can now describe Step (1) in Algorithm 1 in more detail, as Algorithm 2.

Algorithm 2: Step (1) in Algorithm 1

Require: n_0, n, k and f are positive integers, $n_0 < 2^k$.

- (1) if n_0 is excluded by the Descent Sieve, the Path-Merging Sieve, or the Odd-Even-Even Sieve, stop;
 - (2) if $k = M$, output (n_0, n, f) and then stop;
 - (3) if n is even, call Algorithm 2 with $(n_0, n/2, k + 1, f)$ and with $(n_0 + 2^k, (3(n + 3^f) + 1)/2, k + 1, f + 1)$;
 - (4) if n is odd, call Algorithm 2 with $(n_0, (3n + 1)/2, k + 1, f + 1)$ and $(n_0 + 2^k, (n + 3^f)/2, k + 1, f)$.
-

We start Algorithm 2 with $(n_0, n, k, f) = (3, 8, 2, 2)$. We always have $n = T^k(n_0)$. The Descent Sieve is a simple comparison. For the Path-Merging Sieve we keep track of the number of even applications of T after the last odd one and use Lemma 2.8 instead of an explicit mod 3 calculation. We also keep track of when the Odd-Even-Even sieve is in play.

4. A RATIONAL APPROXIMATION TO $\frac{\ln(3)}{\ln(2)}$ AND A PRECOMPUTED BITVECTOR

In this section we describe the bitvectors used in Step (2) of Algorithm 1. This consists of one bitvector for looking ahead another B steps, for each value of f coming out of Algorithm 2, plus one bitvector for the mod 9 Preimage Sieve. The bitvector for looking ahead another B steps incorporates the Descent Sieve, the Path-Merging Sieve, and the Odd-Even-Even Sieve. Together with the bitvector for the mod 9 Preimage Sieve that means we use all four sieves in this step, with only a single bitwise **and** operation.

The key idea is to examine the last B bits of $m = T^{N-A}(n)$ to check if $T^k(n)$ goes below n , or joins the path of some $n_1 < n$, for some $N - A < k \leq N - A + B$. These last bits of m determine the next B steps in the sequence of even and odd applications of T , starting from m .

Under mild conditions on n and k we only need to know $f = f_{N-A}(n)$ and the last B bits of m to determine if this happens.

For ease of computation we use a rational approximation to $\frac{\ln(3)}{\ln(2)}$. We have $\frac{485}{306} > \frac{\ln(3)}{\ln(2)}$, with $\frac{485}{306} - \frac{\ln 3}{\ln 2} = 0.000004819 \dots$. It follows that 3^{306} is slightly smaller than 2^{485} , with $\frac{3^{306}}{2^{485}} = 0.9989 \dots$

Theorem 4.1. *Suppose $T^k(n)$ has $f = f_k(n)$ odd applications and $485f \leq 306k$. Suppose also that $T^m(n) \geq 99781$ for all $0 \leq m \leq k$. Then $T^k(n) < n$.*

Proof. Each odd application increases the number by a factor of at most $\frac{3}{2}(1 + \frac{1}{3.99783})$, so $T^k(n)$ increases by a factor of at most $\frac{3^f}{2^k}(1 + \frac{1}{3.99783})^f < 1$. $\square$

The requirement of $T^m(n) \geq 99781$ for all $0 \leq m \leq k$ can be relaxed, but we leave the details to the reader since we do not need a better bound.

One can obviously prove a similar result for an even closer approximation to $\frac{\ln(3)}{\ln(2)}$, but the added benefit is very small.

We will take it for granted that the Collatz Conjecture has been verified for all $n \leq 99781$, so this means that for a given n we can abort the search as soon as we have computed $T^k(n)$ and we find that $485f_k(n) \leq 306k$. In fact we do not even need to compute $T^k(n)$ if we can compute $f_k(n)$.

The same applies to the Path-Merging Sieve and the Odd-Even-Even sieve.

4.1. Some bitvectors. For each $m \in \{0, 1, \dots, 2^B - 1\}$ we compute the sequence of even and odd applications in $T^B(m)$.

Definition 4.2. Define a set $S'_B(m)$ of integers by taking the union over $k \in \{0, \dots, B\}$ of $306k - 485f_k(m)$. Then define $\text{dip}'_B(m) = \max_{s' \in S'}(s)$.

The significance of $\text{dip}'_B(m)$ is that for any m for which $T^k(m) \geq 99781$ for all $0 \leq k \leq B$, $T^k(m) \leq 2^{-\text{dip}'_B(m_0)/306}m$ for some $0 \leq k \leq B$, so this is useful for the Descent Sieve. We can also incorporate the Path-Merging Sieve and the Odd-Even-Even Sieve:

Definition 4.3. Define a set $S_B(m)$ of integers by taking the union over $k \in \{0, \dots, B\}$ of $306k - 485f_k(m)$ and also the following numbers: If $T^k(m) \equiv 2 \pmod 3$, include $306(k-1) - 485(f_k(m)-1)$. If $T^k(m)$ is followed by a positive number of odd steps and then 2 even steps, include $306(k+1) - 485f_k(m)$.

Then define $\text{dip}_B(m) = \max_{s \in S}(s)$.

It follows that for any m for which $T^k(m)$ is large for all $0 \leq k \leq B$, either the path of m dips below $2^{-\text{dip}_B(m)/306}m$ or it joins the path of some $m_1 < 2^{-\text{dip}_B(m_0)/306}m$. In either case, we can exclude n if $2^{-\text{dip}_B(m_0)/306}m < n$.

Now suppose we start with some n and a calculation of $m = T^{N-A}(n)$. Suppose we also know $f = f_{N-A}(n)$. Then we can use $\text{dip}_B(m)$ to check if $T^k(m) < n$ for some $0 \leq k \leq B$, or equivalently if $T^k(n) < n$ for some $N-A \leq k \leq N-A+B$. We then have the following:

Theorem 4.4. Let $m = T^{N-A}(n)$, let $f = f_{N-A}(n)$, and suppose $\text{dip}_B(m) \geq 485f - 306(N-A)$. Suppose also that $T^m(n) \geq 99781$ for all $0 \leq m \leq N-A+B$. Then either $T^k(n) < n$ for some $N-A \leq k \leq N-A+B$ or the path of n joins the path of some $n' < n$.

Because $\text{dip}_B(m) \geq 0$ for any m , Theorem 4.4 contains Theorem 4.1 as a special case.

There is a small chance that $T^k(n) < n$ even if $\text{dip}_B(m) < 485f - 306(N-A)$, but this can only happen when k is large and we will ignore it. (If this happens we end up checking n in Step (3) of Algorithm 1 even though we do not have to.)

If the conditions in Theorem 4.4 hold, the path of n joins the path of some $n' < n$ and does not need to be considered.

For a fixed f , suppose $f = f_{N-A}(n)$. Then we must have $485f > 306(N-A)$, or $f \geq \lceil \frac{306(N-A)+1}{485} \rceil$. Here $\lceil - \rceil$ denotes the ceiling function, which rounds up to the nearest integer. We will consider a bitvector BV_i of length 2^B corresponding to $f = \lceil \frac{306(N-A)+1}{485} \rceil + i$ for $i \geq 0$.

For example, suppose $N = 72$ and $A = 6$ so $N - 6 = 66$. Then we consider one bitvector BV_i corresponding to $f = 42 + i$ for each $i \geq 0$.

In this example, if n is so that $m = T^{66}(n)$ and $f_{66}(n) = 42$ it follows that if $\text{dip}_B(m) \geq 485 \cdot 42 - 66 \cdot 306 = 174$ then $T^k(n) < n$ for some $66 < k \leq 66 + B$. We prepare the bitvector BV_0 accordingly, setting one bit for every $m \in \{0, 1, \dots, 2^B - 1\}$ with $\text{dip}_B(m) < 174$.

See Figure 4.1 for how effective sieving with BV_i is for the above example. We see that if i is small then this sieve is very effective while i is large it has almost no effect. In the CPU implementation we choose to limit ourselves to $i < 8$ and

Density of bits set for BV_0	0.0198687911
Density of bits set for BV_1	0.0947877764
Density of bits set for BV_2	0.2287583947
Density of bits set for BV_3	0.4023408889
Density of bits set for BV_4	0.5846776366
Density of bits set for BV_5	0.7447901368
Density of bits set for BV_6	0.8630392551
Density of bits set for BV_7	0.9365259408

FIGURE 4.1. The fraction of bits set in each bit vector BV_i for $N = 72$, $A = 6$ and $B = 24$ for $0 \leq i \leq 7$.

run the remaining cases without these sieves, while in the GPU implementation we chose to limit ourselves to $i < 16$ and run the very small number of remaining cases separately without this sieve on the CPU.

To speed up the computation, we do not order the bits in the bitvector for BV_i linearly according to the last 24 bits in m . Instead, if BV_i corresponds to f then bit number s of BV_i corresponds to $s3^f \bmod 2^B$. We do this so that the entries for $T^{N-A}(n)$ and $T^{N-A}(n + 2^{N-A})$ are consecutive. Hence the entries for $T^{N-A}(n + a2^{N-A})$ for $a = 0, 1, \dots, 2^A - 1$ occupy 2^A adjacent entries in the bitvector.

4.2. A bitvector for the mod 9 Preimage Sieve. We refer once again to [3, Section 3.2]. Because $T(2n + 1) = 3n + 2$, any $n \equiv 2 \pmod 3$ does not have to be considered because it is in the path of a smaller number. Similarly, $T^3(8n + 3) = 9n + 4$, so any $n \equiv 4 \pmod 9$ does not have to be considered. Together this removes $4/9$ of all the cases. As discussed in op. cit., using a mod 3^k Preimage Sieve for larger k has limited additional benefit, so we stick with the mod 9 Preimage Sieve.

Now we can complete our discussion of Step (2) of Algorithm 1. Given some $f = f_{N-A}(n_0)$ for $n_0 < 2^{N-A}$, we use the precomputed bitvector BV_i from above together with a precomputed bitvector for the mod 9 Preimage Sieve to indicate for which a the mod 9 value of $n = n_0 + a2^{N-A}$ is allowed, for $a = 0, \dots, 2^A - 1$.

Now we can use a single bitwise **and** operation of the two bitvectors to determine for which $a \in \{0, 1, \dots, 2^A - 1\}$ the number $n = n_0 + a2^{N-A}$ survives all the sieves.

5. STEP (3) OF ALGORITHM 1

The output of Step (2) of Algorithm 1 is a set of numbers $n = n_0 + a2^{N-A}$ which need to be checked manually.

For our CPU implementation we used Lemma 2.3 to do 16 steps at a time until $T^k(n)$ falls below n . The number of steps was dictated in part by the fact that the information needed to do 16 steps can be stored in 32 bits, while the information needed to do 17 steps cannot (at least not easily). This sped up the program by a bit more than a factor of 2, compared to using the same algorithm as the GPU version.

For our GPU implementation we used [3, Algorithm 2], which alternates between doing several odd applications of T in a row and several even applications of T in a row until $T^k(n)$ falls below n .

N	cases to check	percentage to check
35	36 585 292	0.1065%
36	66 320 448	0.0965%
37	125 234 819	0.0911%
38	242 036 747	0.0881%
39	458 345 644	0.0834%
40	888 966 177	0.0806%
41	1 655 180 985	0.0753%
42	3 123 042 856	0.0710%
43	6 041 364 645	0.0687%
44	11 029 246 002	0.0627%
45	20 907 532 072	0.0594%

FIGURE 5.1. The percentage of all numbers to check for $N = 35, \dots, 45$.

5.1. Some empirical evidence. In Figure 5.1 we record how many integers need to be checked for each N for $N = 35, \dots, 45$ using this method. Here $A = 6$ and $B = 24$, and we only used 8 bitvectors $BV_0, \dots, BV_7$.

From this data it looks like considering one additional bit leads to about 1.9 times as many cases. If the trend line holds, that means we have to consider approximately 0.015% of all numbers up to 2^N when $N = 72$ and $A = 6$, or 0.018% when $A = 10$.

We can compare this to [5, Table 3], where the authors conclude that using a size 2^{37} sieve leaves 0.70% of numbers to be considered. Hence we get an improvement of about a factor of about 40 compared to their method.

6. SOME COMMENTS ON THE GPU IMPLEMENTATION

For the GPU implementation, we chose to do Step (1) of Algorithm 1 on the CPU and Steps (2) and (3) on the GPU. We prepare the necessary bitvectors on the CPU, and this takes about 2 seconds.

This minimises the amount of data that has to be transferred from main memory to GPU memory. The bitvectors only have to be moved once, and then we process 2^{16} values of $n_0 < 2^{N-A}$ with the same f at the same time, with each GPU thread responsible for one n_0 . Because all the cases have the same f , this means each thread ends up being responsible for checking a similar number of cases $n = n_0 + a2^{N-A}$ and each thread runs for a similar amount of time. Typically this might be about 20 cases for the smallest possible f and about 500 for a larger f .

We are of course free to choose the constants A and B . Some benchmarking suggested $A = 10$ and $B = 24$ are good choices.

If all we care about is minimising the number of cases to be analysed in Step (3) then choosing $A = 0$ and the largest possible B for which the bitvectors fit in memory is optimal. But it is a balancing act, for several reasons:

- (1) Step (1) is relatively slow. It has branching, recursion, and arithmetic operations on 128-bit integers.
- (2) Step (2) is very fast except for two bottlenecks: We need to compute a mod 9 value, and we need to look up at least one bit in a big bitvector. With $B = 24$ most of the bitvector lives in the L3 cache (or the equivalent for

the GPU) while if $B > 24$ memory access is slower. If we were to look up just one bit, for a single case, then the mod 9 calculation and the latency from looking up that one bit in a bitvector would dominate. But if we look up 2^A bits we treat 2^A cases at the same time while only paying for the mod 9 calculation and the latency once.

- (3) The data generated in Step (1) has to be transferred from main memory to GPU memory for processing in Steps (2) and (3). If $A = 0$ then that is a lot of data to move around, while if $A = 10$ it is an almost trivial amount of data.

Of course the choice of A and B that is optimal on one computer might not be optimal on a different computer.

7. CONCLUSION

We claim that the algorithm presented in this paper is much faster than other algorithms discussed in the literature, both because we end up with fewer cases to consider and because some of the work can be done very effectively using bitwise operations.

Our current computer implementation can almost certainly be optimised, but it is already about two orders of magnitude faster than older programs.

We hope to report on the verification of convergence for $n < 2^N$ for some $N \geq 72$ in future work.

REFERENCES

- [1] David Barina. Path records. <https://pcbarina.fit.vutbr.cz/path-records.htm>. Accessed: 2025-12-05.
- [2] David Barina. Convergence verification of the collatz problem. *The Journal of Supercomputing*, 77(3):2681–2688, 2021.
- [3] David Barina. Improved verification limit for the convergence of the Collatz conjecture. *J Supercomput*, 81, 2025.
- [4] Christian Hercher. There are no Collatz m -cycles with $m \leq 91$. *J. Integer Seq.*, 26(3):Art. 23.3.5, 22, 2023.
- [5] Takumi Honda, Yasuaki Ito, and Koji Nakano. Gpu-accelerated exhaustive verification of the collatz conjecture. *International Journal of Networking and Computing*, 7(1):69–85, 2017.
- [6] Tomás Oliveira e Silva. Maximum excursion and stopping time record-holders for the $3x + 1$ problem: computational results. *Math. Comp.*, 68(225):371–384, 1999.

MATHEMATICAL SCIENCES INSTITUTE, AUSTRALIAN NATIONAL UNIVERSITY, CANBERRA, ACT 0200, AUSTRALIA